
RAG-RAIL: A RETRIEVAL-AUGMENTED GENERATION SYSTEM FOR RAILWAY DOCUMENT QUESTION ANSWERING

A PREPRINT

✉ **Marius Dragic**

R&D Department, Hitachi Rail GTS
CentraleSupélec University
marius.dragic@student-cs.fr

✉ **Nicolas Chauveau**

R&D Department, Hitachi Rail GTS
IMT Atlantique University
nicolas.chauveau@urbanandmainlines.com

Friday, May 24, 2024

ABSTRACT

We introduce RAG-Rail, a domain-specialized Retrieval-Augmented Generation (RAG) system designed to answer technical questions over railway documentation. The system bridges a local language model (via Ollama) with a FAISS-based semantic index of railway documents, enabling fact-grounded and cited answers in French and English. We present its theoretical foundation, pipeline design, and empirical evaluation on retrieval precision and factual correctness. Beyond its technical implementation, RAG-Rail demonstrates how lightweight, local AI can make railway knowledge, such as signaling, safety, and perception, directly accessible to engineers and researchers, thus enhancing traceability and decision-making in safety-critical domains.

Keywords Retrieval-Augmented Generation · Question Answering · Railway Safety · Vector Search · Domain-Specific QA

1 Introduction

The railway industry produces an immense quantity of documentation, from international standards (EN, IEC, ISO) to academic research and industrial white papers, that underpins safety, signaling, and AI-based perception systems. Engineers, certification teams, and R&D units must frequently consult these resources to address questions ranging from cybersecurity requirements to autonomous driving constraints. However, most of these documents remain buried in large PDF collections, with inconsistent formats and minimal searchability.

Large Language Models (LLMs) are promising for knowledge access, yet their inability to reference verifiable sources or their tendency to hallucinate makes them unsuitable for safety-critical use. Retrieval-Augmented Generation (RAG) offers a viable alternative by coupling language generation with information retrieval. Rather than relying solely on internal model weights, RAG retrieves factual excerpts from a trusted corpus before formulating an answer.

In this paper, we propose **RAG-Rail**, a local and domain-adapted RAG architecture for railway document understanding. It combines a FAISS vector index for semantic retrieval with a local LLM (Mistral 7B via Ollama) to generate concise and referenced answers. Our objective is not only to design an operational backend, but to validate the conceptual foundations and evaluate the effectiveness of RAG on a technical and multilingual corpus.

2 Related Work

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. [1] as a framework to ground language model outputs in retrieved textual evidence, improving factual accuracy in knowledge-intensive tasks. Subsequent works and surveys have established RAG as a key paradigm for mitigating hallucinations in large language models, and have explored dense, sparse, and hybrid retrieval strategies, as well as evaluation challenges in real-world deployments [2, 3].

In applied domains, the use of RAG for technical and transportation-related document analysis is emerging but remains limited. Recent studies have investigated retrieval-augmented question answering systems for transport and railway-related documentation, demonstrating the potential of domain-specific retrieval to support operational and technical queries [4, 5]. However, these approaches are generally exploratory and do not explicitly address the heterogeneity, traceability, and safety-critical constraints of railway standards and engineering documents. RAG-Rail addresses this gap by proposing a domain-adapted, local RAG system specifically designed for railway documentation.

3 Theoretical Background

RAG-Rail builds upon two main theoretical pillars: vector-based semantic retrieval and conditional text generation.

Vector Semantic Search

Given a corpus of text chunks $\{d_i\}_{i=1}^N$, each document d_i is mapped to a dense vector $\mathbf{v}_i = f_\theta(d_i)$ using an embedding model f_θ trained to capture semantic similarity. A user query q is similarly encoded as $\mathbf{v}_q = f_\theta(q)$. Retrieval consists of selecting the top- k documents that maximize the cosine similarity:

$$R_k(q) = \arg \max_{d_i} \frac{\mathbf{v}_q \cdot \mathbf{v}_i}{\|\mathbf{v}_q\| \|\mathbf{v}_i\|}.$$

These retrieved contexts are then injected into the generation prompt.

Retrieval-Augmented Generation

The generative model G_ϕ conditions on both the query q and the retrieved context $C = \{d_i\}_{i \in R_k(q)}$:

$$p_\phi(y|q, C) = \prod_{t=1}^T p_\phi(y_t | y_{<t}, q, C),$$

where y_t are output tokens. This conditioning constrains generation to grounded information while maintaining the fluency of natural language.

This dual mechanism, retrieval for factual grounding and generation for linguistic synthesis, provides the theoretical backbone for RAG-Rail.

4 System Design

4.1 Corpus Acquisition and Preprocessing

Railway-related documents were collected from multiple sources, including arXiv papers, open standards, and safety reports. A lightweight scraper continuously polls relevant RSS feeds and filters documents containing keywords such as *railway*, *signalling*, and *autonomous train*. After parsing via pypdf, texts are segmented into overlapping windows of 300 to 600 tokens using the LangChain splitter. Each segment retains metadata including filename and page number, enabling fine-grained traceability.

4.2 Embedding and Retrieval

Each chunk is encoded with the all-MiniLM-L6-v2 sentence-transformer model into a 384-dimensional vector space. All embeddings are indexed using FAISS for high-speed approximate search. Queries are embedded in the same space, and the nearest chunks are retrieved according to cosine similarity.

Retrieval is followed by a lightweight lexical re-ranking stage using BM25 to enhance robustness for queries containing domain-specific acronyms (e.g., SIL, ERTMS).

4.3 Generation Model and Inference Backend

For generation, RAG-Rail relies on a local instance of the *Mistral* language model served via Ollama. The model corresponds to a 7.2B-parameter architecture with a 32k token context window, using a GGUF quantization scheme

(Q8_0) to leverage the available GPU memory and maximize generation fidelity. This configuration enables fully local, low-latency inference while providing sufficient capacity to handle long retrieved contexts in retrieval-augmented question answering over technical railway documents. Relying instead on the Mistral API to access larger-scale models would likely yield higher generation quality and reasoning capabilities, at the cost of increased operational expenses and reduced deployment autonomy.

4.4 Prompting and Citation Strategy

To ensure interpretability and verifiability, RAG-Rail explicitly embeds the retrieved chunks in the final prompt given to the LLM. Each passage is numbered and cited in the final answer. The prompt template enforces both structure and honesty, instructing the model to cite every statement and abstain from speculation.

Prompt Template Example

Context:

(source: IEC_62443_Assessment.pdf, page 3): “Industrial control systems must ensure cybersecurity across all levels.”

(source: Railway_Safety_Framework.pdf, page 7): “IEC 62425 defines safety-related communication standards.”

Question: What cybersecurity standards are used in railway signalling?

Instruction: Answer concisely in French, citing each source using [1], [2], etc. If no sufficient context exists, say so.

This structured prompting notably reduces hallucination rates and improves user trust, aligning with recent best practices in grounded LLM reasoning.

5 Experimental Evaluation

Experiments were conducted on a corpus of 200 railway-related PDFs, producing around 5,000 chunks. The local inference backend used `mistral` served via Ollama on a workstation with an NVIDIA RTX A4500 GPU (20 GB VRAM). The evaluation covered retrieval accuracy, citation consistency, and latency.

Quantitatively, retrieval achieved a Recall@5 of 0.89, while 92% of generated answers correctly referenced existing chunks. Average response time was approximately 450 ms. These figures confirm that a lightweight local RAG pipeline can achieve both responsiveness and factual precision on a constrained domain.

Table 1: Evaluation metrics for RAG-Rail.

Metric	Value	Unit
Average retrieval time	21	ms
Average generation time	430	ms
Recall@3	0.82	
Recall@5	0.89	
Citation precision	0.92	

Qualitative inspection further confirmed the system’s reliability. When asked, for example, “*Which standards regulate safety integrity levels in railway control systems?*”, RAG-Rail produced a correct and traceable answer referencing IEC 61508 and IEC 62425, while noting that context about SIL classification was limited to extracted excerpts.

These results indicate that RAG-Rail effectively balances conciseness, factual grounding, and interpretability, an essential triad in technical question answering.

6 Applications and Discussion

Beyond its experimental success, RAG-Rail opens a path toward accessible and transparent AI assistance within industrial and research environments. By centralizing thousands of heterogeneous documents into a searchable

knowledge base, it becomes possible for engineers to rapidly query specific standards, safety requirements, or perception algorithms. Certification teams could verify compliance clauses automatically, while R&D staff gain a powerful tool for literature exploration.

Several challenges remain. Domain-specific embedding fine-tuning could significantly improve retrieval quality for niche terminology (e.g., "SIL", "IEC" etc.). Hybrid search, combining dense embeddings and symbolic reasoning, would also strengthen contextual relevance. Moreover, the incorporation of multimodal inputs, such as figures, tables, and diagrams, could provide richer factual grounding, essential in technical documentation.

7 Conclusion

We presented RAG-Rail, a retrieval-augmented question-answering system dedicated to railway documentation. By integrating FAISS-based retrieval with a local language model, it delivers concise, grounded, and cited responses suitable for safety-critical domains. Experimental validation demonstrates high recall and citation precision, proving that a fully local RAG pipeline can achieve reliable performance without relying on cloud APIs. In future work, we plan to extend RAG-Rail with domain-adapted embeddings, multimodal context retrieval, and explainability layers linking confidence scores to citation relevance.

References

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- [2] Yunfan Gao et al. “Retrieval-Augmented Generation for Large Language Models: A Survey”. In: *arXiv preprint arXiv:2312.10997* (2023). URL: <https://arxiv.org/abs/2312.10997>.
- [3] Ori Ram et al. “In-Context Retrieval-Augmented Language Models”. In: *Transactions of the ACL* (2023). URL: <https://arxiv.org/abs/2302.00083>.
- [4] Anonymous. “Intelligent Railways: Leveraging Retrieval-Augmented Generation for Smarter Systems”. In: *International Conference on Transportation and AI Systems*. 2024. URL: <https://internationalpubls.com/index.php/cana/article/download/2553/1552/4585>.
- [5] Anonymous. *Open-Domain QA System Using RAG for Swedish Transport Documents*. 2024. URL: <https://du.diva-portal.org/smash/get/diva2%3A1889885/FULLTEXT01.pdf>.